

Terceiro projeto - Introdução à Física Computacional

Evandro Risso Venâncio¹

¹Instituto de Física de São Carlos - IFSC USP

risso.evandro@usp.br

Introdução

Este relatório tem como objetivo principal apresentar e explicar os códigos desenvolvidos no terceiro projeto da matéria de Introdução à Física Computacional, além de discutir brevemente alguns conceitos básicos de cálculo numérico, como os métodos de derivação, integração numérica e métodos para estimativa de raízes de uma função.

Exercício 1

O primeiro exercício tem como objetivo calcular as derivadas de $f(x)$ em $x = 1/2$ utilizando aproximações de diferentes ordens. A função $f(x)$ é:

$$f(x) = e^{x^2} \cot(2x)$$

A derivação numérica é feita com fórmulas de diferenças finitas, e os resultados foram comparados com os valores exatos da primeira e segunda derivada.

A fórmula de dois pontos para frente utiliza os valores da função $f(x)$ no ponto x e no ponto $x + h$ para estimar a derivada:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

Essa fórmula utiliza um ponto à frente de x . Quanto menor o valor de h , mais precisa será a aproximação. No entanto, h muito pequeno pode introduzir erros de arredondamento. A 'diferença para trás' utiliza o valor de $f(x)$ e o ponto imediatamente anterior $f(x-h)$:

$$f'(x) \approx \frac{f(x) - f(x-h)}{h}$$

Essa fórmula também depende de h para sua precisão.

A fórmula de três pontos simétrica é mais precisa que as fórmulas anteriores, pois utiliza uma combinação de pontos à frente e atrás de x :

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

A simetria dessa fórmula faz com que o erro de truncamento seja menor, já que ela considera as mudanças em $f(x)$ de maneira mais balanceada ao redor do ponto central x .

Para aumentar ainda mais a precisão, a fórmula de cinco pontos simétrica, leva em consideração os valores de $f(x)$ em $x-2h$, $x-h$, $x+h$ e $x+2h$:

$$f'(x) \approx \frac{f(x-2h) - 8f(x-h) + 8f(x+h) - f(x+2h)}{12h}$$

Essa fórmula de ordem superior oferece maior precisão ao incluir mais pontos ao redor de x , diminuindo o erro de truncamento, especialmente para valores intermediários de h . Para estimar a segunda derivada de $f(x)$, foi utilizado uma fórmula simétrica de três pontos, que calcula a segunda derivada como:

$$f''(x) \approx \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}$$

Essa fórmula calcula a curvatura da função em torno de x , observando a diferença entre os valores de $f(x)$ nos pontos $x+h$ e $x-h$.

Para uma estimativa ainda mais precisa da segunda derivada, foi utilizado uma fórmula de cinco pontos:

$$f''(x) \approx \frac{-f(x-2h) + 16f(x-h) - 30f(x) + 16f(x+h) - f(x+2h)}{12h^2}$$

Assim como a fórmula de cinco pontos para a primeira derivada, essa aproximação de segunda derivada usa mais pontos ao redor de x , o que reduz o erro de truncamento.

As fórmulas de ordens superiores, como as de cinco pontos, são mais precisas porque consideram mais pontos ao redor do valor de interesse x . No entanto, quando h é muito pequeno, o erro de arredondamento começa a influenciar os resultados, tornando essas aproximações menos precisas.

As aproximações simétricas, como a fórmula de três pontos e a de cinco pontos, oferecem maior precisão do que as aproximações de dois pontos, pois elas avaliam a função de forma mais equilibrada, levando em consideração tanto os pontos à frente quanto os pontos atrás de x .

O código está listado abaixo. Ele calcula as derivadas numéricas para diferentes valores de h , que variam de 5^{-1} até 5^{-12} .

```

1  program ex1
2      real*8 x, h, f, f2f, f2t, f3s, f5s, ff3s, ff5s, fExato, f2Exato
3
4      x = 0.5
5      write(1, 110) "h", "f'2s", "f'2t", "f'3s", "f'5s", "f''3s", "f''5s"
6      write(1, 120)
7
8      fExato = -2.80235111004d0
9      f2Exato = 4.53476386947d0
10
11     do i = 1, 12
12         h = 5.0d0 ** (-i)
13
14         f2f = (f(x+h) - f(x)) / h
15
16         f2t = (f(x) - f(x-h)) / h
17
18         f3s = (f(x+h) - f(x-h)) / (2*h)
19
20         f5s = (f(x-(2*h)) - 8*f(x-h) + 8*f(x+h) - f(x+(2*h))) / (12*h)
21
22         ff3s = (f(x+h) - 2*f(x) + f(x-h)) / (h**2)
23
24         ff5s = -f(x-(2*h)) + 16*f(x-h) - 30*f(x) + 16*f(x+h) - f(x+(2*h))

```

```

25      ff5s = ff5s / (12*(h**2))
26
27      write(1, 100) h, f2f, f2t, f3s, f5s, ff3s, ff5s
28  end do
29
30  write(1, 120)
31  write(1, 13) "Exato", fExato, fExato, fExato, fExato, f2Exato, f2Exato
32
33 100  format(f20.12, 9("|", f20.11), "|")
34 110  format(a20, 9("|", a20), "|")
35 120  format(147("-"))
36 13   format(a20, 20("|", f20.11), "|")
37
38  end program ex1
39
40  real*8 function f(x)
41      real*8 x
42      f = exp(x**2)*(1/tan(2*x))
43  end function f

```

Os valores exatos das derivadas foram calculados e inseridos no código para comparação (*fExato* e *f2Exato*).

Os resultados obtidos para as diferentes aproximações numéricas estão apresentados na Tabelas 1 e 2.

h	$f'2s$	$f'2t$	$f'3s$
0.20000000000000	-2.71463340794	-3.87443438581	-3.29453389687
0.04000000000000	-2.72793570106	-2.91098965954	-2.81946268030
0.00800000000000	-2.78488640648	-2.82117772872	-2.80303206760
0.00160000000000	-2.79875047884	-2.80600620669	-2.80237834277
0.00032000000000	-2.80162663670	-2.80307776198	-2.80235219934
0.00006400000000	-2.80220604116	-2.80249626606	-2.80235115361
0.00001280000000	-2.80232208930	-2.80238013428	-2.80235111179
0.00000256000000	-2.80234530550	-2.80235691458	-2.80235111004
0.00000051200000	-2.80234994867	-2.80235227146	-2.80235111006
0.00000010240000	-2.80235087739	-2.80235134252	-2.80235110995
0.00000002048000	-2.80235105954	-2.80235115712	-2.80235110833
0.00000000409600	-2.80235107038	-2.80235109749	-2.80235108393
Exato	-2.80235111004	-2.80235111004	-2.80235111004

Tabela 1. Resultados das primeiras derivadas numéricas ($f'2s$, $f'2t$, $f'3s$).

h	$f'5s$	$f''3s$	$f''5s$
0.200000000000	-2.09805403514	5.79900488937	1.87916787423
0.040000000000	-2.80197718008	4.57634896200	4.53348077649
0.008000000000	-2.80235052875	4.53641528098	4.53476188201
0.001600000000	-2.80235110911	4.53482990690	4.53476386638
0.000320000000	-2.80235111004	4.53476651195	4.53476387147
0.000064000000	-2.80235111004	4.53476400311	4.53476391050
0.000012800000	-2.80235111005	4.53476367785	4.53476418607
0.000002560000	-2.80235110994	4.53479417103	4.53480828825
0.000000512000	-2.80235111003	4.53670846549	4.53776725668
0.000000102400	-2.80235111032	4.54221417965	4.55103743952
0.000000020480	-2.80235110923	4.76456032831	4.98514182499
0.000000004096	-2.80235107038	6.61744490042	13.23488980085
Exato	-2.80235111004	4.53476386947	4.53476386947

Tabela 2. Resultados das derivadas numéricas ($f'5s$, $f''3s$, $f''5s$).

Ao comparar os resultados, podemos observar que conforme h diminui, os valores das aproximações numéricas convergem para os valores exatos. No entanto, para valores muito pequenos de h , o erro de arredondamento começa a influenciar os resultados. A aproximação de ordem 5 se mostrou a mais precisa, com erros menores comparados aos métodos de menor ordem.

Exercício 2

Neste exercício, o objetivo é calcular a integral:

$$I = \int_0^{2\pi} e^{-x} \cos(x) dx$$

utilizando três métodos de quadratura numérica: a regra do trapézio, a regra de Simpson e a regra de Boole. O programa foi implementado em partições sucessivas do intervalo $[0, 2\pi]$, calculando a integral com diferentes tamanhos de passo h e comparando os resultados com o valor exato da integral, que é:

$$I = 0.49906627797$$

No programa principal, foi definido os limites de integração $a = 0$ e $b = 2\pi$, utilizando a constante π calculada com a função `acos(-1.0D0)`. A integral é calculada para diferentes valores de h , que variam conforme a partição do intervalo $[0, 2\pi]$. Para cada partição, foi calculada a integral usando os três métodos (Trapézio, Simpson e Boole) e seus resultados foram exibidos em uma tabela, juntamente com o valor exato da integral. O valor de h é calculado como:

$$h = \frac{b - a}{2^i}$$

onde i é o índice que varia de 2 a 13, correspondendo ao número de subdivisões do intervalo. O valor exato da integral é armazenado na variável `aexato`.

A **regra do trapézio** é uma técnica de quadratura simples. A ideia é aproximar a área sob a curva $f(x)$ ao longo do intervalo $[a, b]$ utilizando trapézios. A fórmula da regra do trapézio para n subintervalos é dada por:

$$I_{\text{trapézio}} \approx h \left(\frac{f(a) + f(b)}{2} + \sum_{i=1}^{n-1} f(a + i \cdot h) \right)$$

onde h é o tamanho de cada subintervalo, dado por $h = \frac{b-a}{n}$, e $x_i = a + i \cdot h$ são os pontos intermediários. A função $f(x)$ é avaliada nos pontos extremos a e b , e em cada ponto intermediário, somando-se os valores obtidos.

A regra do trapézio é uma aproximação de ordem 1, o que significa que seu erro é proporcional a h^2 . Portanto, a precisão aumenta à medida que h diminui, mas o método é menos preciso do que técnicas de ordem superior.

A **regra de Simpson** é um método de quadratura numérica de ordem 3, que oferece maior precisão do que a regra do trapézio. A ideia básica é aproximar a função $f(x)$ por uma parábola entre cada três pontos consecutivos e calcular a área sob a parábola.

A fórmula aplicada para n subintervalos (onde n deve ser par) é:

$$I_{\text{Simpson}} \approx \frac{h}{3} \left(f(a) + f(b) + 4 \sum_{\text{ímpar}} f(x_i) + 2 \sum_{\text{par}} f(x_i) \right)$$

onde $h = \frac{b-a}{n}$ e $x_i = a + i \cdot h$. Os coeficientes 4 e 2 são aplicados alternadamente aos pontos intermediários: 4 para os pontos ímpares e 2 para os pontos pares. Assim, a função $f(x)$ é avaliada em a , b , e nos pontos intermediários, atribuindo diferentes pesos conforme sua posição.

No código, a função `simpson(a, ab, ah)` implementa essa fórmula. O valor de h e os valores de $f(x)$ nos pontos intermediários são somados com os respectivos pesos (4 para os índices ímpares e 2 para os pares).

A regra de Simpson é mais precisa que a regra do trapézio, pois o erro é proporcional a h^4 , o que significa que a precisão aumenta significativamente para intervalos pequenos.

A **regra de Boole** é uma técnica de quadratura de ordem ainda superior, utilizando polinômios de grau 4 para aproximar a função $f(x)$ em cada subintervalo. Esse método se baseia em dividir o intervalo $[a, b]$ em partes e aplicar uma combinação ponderada dos valores da função nos pontos extremos e intermediários.

A fórmula da regra de Boole para n subintervalos (onde n deve ser múltiplo de 4) é:

$$I_{\text{Boole}} \approx \frac{2h}{45} \left(7(f(a) + f(b)) + 32 \sum_{\text{ímpar}} f(x_i) + 12 \sum_{\text{par}} f(x_i) \right)$$

Neste método, os pontos extremos a e b recebem o peso 7, os pontos ímpares recebem o peso 32, e os pontos pares recebem o peso 12. Esses coeficientes são ajustados para maximizar a precisão do método.

A regra de Boole tem erro proporcional a h^6 , sendo muito mais precisa que os métodos anteriores.

O código completo da implementação destes métodos está listado abaixo:

```
1 program ex2
```

```

2      real*8 ah, api, a, ab, atrap, asimp, aboo, aexato
3      real*8 trapezio, simpson, boole
4
5      api = acos(-1.0D0)
6      a = 0.0D0
7      ab = 2.0D0*api
8
9      aexato = 0.49906627797d0
10
11     write(1, 110) "2*pi X h^-1", "Trapezio", "Simpson", "Boole"
12     write(1, 120)
13
14     do i = 2, 13
15         ah = (ab - a) / (2.0d0 ** i)
16         in = 2 ** i
17
18         atrap = trapezio(a, ab, ah)
19         asimp = simpson(a, ab, ah)
20         aboo = boole(a, ab, ah)
21
22         write(1, 100) in, atrap, asimp, aboo
23     end do
24
25     write(1, 120)
26     write(1, 13) "Exato", aexato, aexato, aexato
27
28 100    format(i15, 4("|", f18.11), "|")
29 110    format(a15, 4("|", a18), "|")
30 120    format(73("-"))
31 13     format(a15, 4("|", f18.11), "|")
32 end program ex2
33
34 real*8 function trapezio(a, ab, ah)
35     real*8 a, ab, ah, asoma, f
36     integer in
37
38     in = int((ab - a) / ah)
39     asoma = (f(a) + f(ab)) * 0.5d0
40
41     do i = 1, in - 1
42         asoma = asoma + f(a + i*ah)
43     end do
44
45     trapezio = asoma * ah
46 end function trapezio
47
48 real*8 function simpson(a, ab, ah)
49     real*8 a, ab, ah, asoma, f
50     integer in
51
52     in = int((ab - a) / ah)
53     asoma = f(a) + f(ab)
54
55     do i = 1, in - 1
56         if (mod(i, 2) == 0) then
57             asoma = asoma + 2.0d0 * f(a + i*ah)

```

```

58         else
59             asoma = asoma + 4.0d0 * f(a + i*ah)
60         end if
61     end do
62
63     simpson = asoma * ah / 3.0d0
64 end function simpson
65
66
67 real*8 function boole(a, ab, ah)
68     real*8 a, ab, ah, asoma, f
69     integer in
70
71     in = int((ab - a) / ah)
72     asoma = 0.0d0
73
74     do i = 2, in - 2, 4
75         asoma = asoma + 7.0d0 * (f(a + (i-2)*ah) + f(a + (i+2)*ah))
76         asoma = asoma + 32.0d0 * (f(a + (i-1)*ah) + f(a + (i+1)*ah))
77         asoma = asoma + 12.0d0 * f(a + i*ah)
78     end do
79
80     boole = asoma * 2.0d0 * ah / 45.0d0
81 end function boole
82
83 real*8 function f(ax)
84     real*8 ax
85     f = exp(-ax)*cos(ax)
86 end function f

```

A regra do trapézio no código tem a seguinte abordagem:

- O valor inicial de `asoma` é a média ponderada dos valores da função $f(x)$ nos pontos extremos a e b , ou seja, $(f(a) + f(b))/2$.
- Em seguida, o laço `do` percorre os pontos intermediários $x_i = a + i \cdot h$, somando $f(x_i)$ ao valor acumulado em `asoma`.
- Finalmente, o valor da integral é obtido multiplicando o valor acumulado por h (o passo entre os pontos), resultando em `trapezio = asoma * ah`.

Para a regra de Simpson temos a seguinte abordagem:

- O valor inicial de `asoma` é a soma dos valores da função $f(x)$ nos pontos extremos a e b , ou seja, $f(a) + f(b)$.
- Em seguida, o laço `do` percorre todos os pontos intermediários $x_i = a + i \cdot h$:
 - Se o índice i for ímpar ($\text{mod}(i, 2) == 1$), o valor de $f(x_i)$ é multiplicado por 4 e somado ao valor acumulado em `asoma`.
 - Se o índice i for par ($\text{mod}(i, 2) == 0$), o valor de $f(x_i)$ é multiplicado por 2 e somado ao valor acumulado em `asoma`.
- Finalmente, o valor da integral é obtido multiplicando o valor acumulado por $h/3$, resultando em `simpson = asoma * ah / 3.0d0`.

E por último para a regra de Boole:

- A variável `asoma` é inicializada em zero e, em seguida, o laço `do` percorre os pontos intermediários $x_i = a + i \cdot h$, indo de quatro pontos por vez.

- Para cada iteração, os valores de $f(x)$ em quatro pontos são somados com diferentes pesos:
 - O peso 7 é aplicado a dois pontos adjacentes, $f(a + (i - 2) \cdot h)$ e $f(a + (i + 2) \cdot h)$.
 - O peso 32 é aplicado a dois outros pontos intermediários, $f(a + (i - 1) \cdot h)$ e $f(a + (i + 1) \cdot h)$.
 - O peso 12 é aplicado ao ponto $f(a + i \cdot h)$ central do grupo.
- Após somar todos os pontos intermediários com seus respectivos pesos, o valor acumulado em a_{soma} é multiplicado por $\frac{2h}{45}$, resultando na aproximação final da integral.

Os resultados das integrais aproximadas calculadas pelos três métodos estão exibidos na Tabela 3. O valor exato da integral é comparado ao final, e podemos observar a convergência dos resultados conforme o número de subdivisões h diminui.

$2\pi \times h^{-1}$	Trapézio	Simpson	Boole
4	0.71898458541	0.47932305694	0.45340198773
8	0.55139754235	0.49553519467	0.49661600385
16	0.51195877594	0.49881252046	0.49903100885
32	0.50227715428	0.49904994706	0.49906577550
64	0.49986822663	0.49906525075	0.49906627099
128	0.49926671737	0.49906621428	0.49906627852
256	0.49911638530	0.49906627461	0.49906627863
512	0.49907880511	0.49906627838	0.49906627863
1024	0.49906941024	0.49906627862	0.49906627863
2048	0.49906706154	0.49906627863	0.49906627863
4096	0.49906647436	0.49906627863	0.49906627863
8192	0.49906632757	0.49906627863	0.49906627863
Exato	0.49906627797	0.49906627797	0.49906627797

Tabela 3. Comparação dos resultados das integrais utilizando os métodos de Trapézio, Simpson e Boole

Exercício 3

O último exercício tem como objetivo encontrar as raízes reais de $f(x)$ no intervalo de $x \in [-10, 10]$. A função $f(x)$ é a seguinte:

$$f(x) = 27x^3 - 522x^2 + 3003x - 4508$$

Os métodos utilizados foram para encontrar as raízes foram:

- Bisseção
- Newton-Raphson
- Secante

O **método da bisseção** é um método para encontrar raízes de uma função contínua $f(x)$. Ele se baseia na ideia de que se uma função contínua $f(x)$ muda de sinal em um intervalo $[a, b]$, isto é, se $f(a) \cdot f(b) < 0$, então existe pelo menos uma raiz x^* nesse intervalo. A ideia central do método é dividir o intervalo $[a, b]$ ao meio em cada iteração e escolher

o subintervalo onde ocorre a mudança de sinal, reduzindo o intervalo de busca pela raiz. A cada iteração, o ponto médio do intervalo $x_{\text{médio}}$ é calculado como:

$$x_{\text{médio}} = \frac{a + b}{2}$$

Em seguida, se $f(a) \cdot f(x_{\text{médio}}) < 0$, a raiz está no intervalo $[a, x_{\text{médio}}]$; caso contrário, está no intervalo $[x_{\text{médio}}, b]$. O processo é repetido até que o intervalo seja suficientemente pequeno ou até que $f(x_{\text{médio}})$ seja próximo o suficiente de zero, dentro de uma tolerância pré-estabelecida.

O **método de Newton-Raphson** é uma técnica que aproveita a derivada da função para refinar a estimativa da raiz. O método parte de uma estimativa inicial x_0 e, em cada iteração, usa a fórmula:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

onde $f'(x_n)$ é a derivada da função no ponto x_n . A ideia por trás do método é que a linha tangente à curva $f(x)$ no ponto x_n intercepta o eixo x em um ponto próximo da raiz. Esse ponto de interseção é tomado como a nova aproximação x_{n+1} , e o processo é repetido.

O **método da secante** é uma variante do método de Newton-Raphson, mas com uma diferença: ele não usa a derivada exata da função. Em vez disso, o método da secante aproxima a derivada utilizando duas aproximações x_n e x_{n-1} da seguinte forma:

$$x_{n+1} = x_n - f(x_n) \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$$

A ideia por trás do método da secante é similar à do método de Newton-Raphson: usa-se a inclinação de uma linha para aproximar a raiz. Contudo, em vez de usar a tangente (derivada exata), o método da secante usa uma inclinação baseada em dois pontos da função, o que fornece uma aproximação da derivada.

Os três métodos têm suas próprias características e vantagens:

- O **método da bisseção** tem uma convergência lenta;
- O **método de Newton-Raphson** é mais rápido, com convergência quadrática;
- O **método da secante** é uma boa alternativa ao método de Newton-Raphson quando a derivada não está disponível, mas ainda assim pode ser menos robusto que a bisseção.

Sabendo da teoria por trás o código a seguir foi produzido:

```
1  program ex3
2      real*8 atolerancia, ax, ah, abis(10), anew(10), asec(10)
3      real*8 f
4
5      atolerancia = 1.0d-6
6      ah = 0.1d0
7      n = int((10.0d0 - (-10.0d0)) / ah)
8      k = 1
9
10     do i = 0, n
11         ax = -10.0d0 + i*ah
12         if (f(ax) * f(ax + ah) .LT. 0.0d0) then
13             call bissecao(ax, ax + ah, atolerancia, abis)
```

```

14      call newton(ax, atolerancia, anew)
15      call secante(ax, ax + ah, atolerancia, asec)
16
17      write(1, 90) "Raiz", k
18      write(1, 110) "Bissecao", "Newton", "Secante", "Exato"
19
20      k = k + 1
21
22      do j = 2, 10
23          write(1, 120) abis(j), anew(j), asec(j), ax
24      end do
25
26      write(1, 130)
27  end if
28 end do
29
30 90      format(a5, i3)
31 110     format(a15, a15, a15, a15)
32 120     format(f15.10, f15.10, f15.10, f15.10)
33 130     format(30("-"))
34 end program ex3
35
36
37 subroutine bissecao(axmenos, axmais, atolerancia, abis)
38     real*8 axmenos, axmais, atolerancia, axmedio, f
39     real*8 abis(10)
40
41     iter = 0
42     imaxiter = 10
43
44     do while (iter .LT. imaxiter)
45         axmedio = (axmenos + axmais) / 2.0d0
46         abis(iter + 1) = axmedio
47
48         if (abs(axmais - axmenos) / 2.0d0 .LE. atolerancia) exit
49
50         if (f(axmenos) * f(axmedio) .LT. 0.0d0) then
51             axmais = axmedio
52         else
53             axmenos = axmedio
54         end if
55
56         iter = iter + 1
57     end do
58
59     do while (iter .LT. 10)
60         abis(iter+1) = axmedio
61         iter = iter + 1
62     end do
63 end subroutine bissecao
64
65 subroutine newton(ax, atolerancia, anew)
66     real*8 ax, atolerancia, axnovo, f, df
67     real*8 anew(10)
68
69     iter = 0

```

```

70     imaxiter = 10
71
72     do while (iter .LT. imaxiter)
73         axnovo = ax - f(ax) / df(ax)
74         anew(iter + 1) = axnovo
75
76         if (abs(f(axnovo)) .LE. atolerancia) exit
77         if (abs(axnovo - ax) .LE. atolerancia) exit
78
79         ax = axnovo
80         iter = iter + 1
81     end do
82
83     do while (iter .LT. 10)
84         anew(iter+1) = axnovo
85         iter = iter + 1
86     end do
87 end subroutine newton
88
89 subroutine secante(axme, axma, atolerancia, asec)
90     real*8 axme, axma, atolerancia, axnovo, f
91     real*8 asec(10)
92
93     iter = 0
94     imaxiter = 10
95
96     do while (iter .LT. imaxiter)
97         axnovo = axme - f(axme) * (axme - axma) / (f(axme) - f(axma))
98         asec(iter + 1) = axnovo
99
100        if (abs(axnovo - axme) .LE. atolerancia) exit
101
102        axma = axme
103        axme = axnovo
104        iter = iter + 1
105    end do
106
107    do while (iter .LT. 10)
108        asec(iter+1) = axnovo
109        iter = iter + 1
110    end do
111 end subroutine secante
112
113 real*8 function f(ax)
114     real*8 ax
115     f = 27*ax**3 - 522*ax**2 + 3003*ax - 4508
116 end function f
117
118 real*8 function df(ax)
119     real*8 ax
120     df = 81*ax**2 - 1044*ax + 3003
121 end function df

```

O código para o método da bissecção de forma resumida:

- O laço `do while` percorre até um máximo de 10 iterações (`imaxiter = 10`).

- Em cada iteração:
 - O ponto médio $x_{\text{médio}}$ do intervalo $[ax_{\text{menos}}, ax_{\text{mais}}]$ é calculado como $\frac{ax_{\text{menos}} + ax_{\text{mais}}}{2}$.
 - A aproximação da raiz $x_{\text{médio}}$ é armazenada no vetor `abis` na posição correspondente à iteração.
 - Se o tamanho do intervalo $(ax_{\text{mais}} - ax_{\text{menos}})/2$ for menor que a tolerância, o laço é encerrado.
 - Se o produto $f(ax_{\text{menos}}) \cdot f(x_{\text{médio}}) < 0$, a raiz está no intervalo $[ax_{\text{menos}}, x_{\text{médio}}]$, e o limite superior (`axmais`) é atualizado para $x_{\text{médio}}$.
 - Caso contrário, a raiz está no intervalo $[x_{\text{médio}}, ax_{\text{mais}}]$, e o limite inferior (`axmenos`) é atualizado para $x_{\text{médio}}$.
- Após o término do laço principal, outro laço preenche o vetor `abis` com o valor final de $x_{\text{médio}}$ nas posições restantes, caso a convergência tenha ocorrido antes das 10 iterações.

Para o método de Newton-Raphson:

- O laço do `while` percorre até um máximo de 10 iterações (`imaxiter = 10`).
- Em cada iteração:
 - A nova aproximação da raiz x_{novo} é calculada pela fórmula de Newton-Raphson:

$$x_{\text{novo}} = x - \frac{f(x)}{f'(x)}$$
 - A nova aproximação x_{novo} é armazenada no vetor `anew` na posição correspondente à iteração.
 - Se $|f(x_{\text{novo}})|$ for menor ou igual à tolerância, o laço é interrompido.
 - Se a diferença entre x_{novo} e x for menor ou igual à tolerância, o laço também é interrompido.
 - Após cada iteração, a variável x é atualizada para x_{novo} .
- Após o término do laço principal, outro laço preenche o vetor `anew` com o valor final de x_{novo} nas posições restantes, caso a convergência tenha ocorrido antes das 10 iterações.

Por último o método da Secante:

- O laço do `while` percorre até um máximo de 10 iterações (`imaxiter = 10`).
- Em cada iteração:
 - A nova aproximação da raiz x_{novo} é calculada pela fórmula do método da secante:

$$x_{\text{novo}} = x_{\text{me}} - f(x_{\text{me}}) \frac{x_{\text{me}} - x_{\text{ma}}}{f(x_{\text{me}}) - f(x_{\text{ma}})}$$
 - O valor x_{novo} é armazenado no vetor `asec` na posição correspondente à iteração.
 - Se a diferença entre x_{novo} e x_{me} for menor ou igual à tolerância, o laço é interrompido.
 - Após cada iteração, as variáveis x_{ma} e x_{me} são atualizadas, onde:
 - * x_{ma} recebe o valor de x_{me} .
 - * x_{me} recebe o valor de x_{novo} .

- Após o término do laço principal, outro laço preenche o vetor `asec` com o valor final de x_{novo} nas posições restantes, caso a convergência tenha ocorrido antes das 10 iterações.

O resultado das raízes para cada método está sendo ilustrado nas Tabelas 4, 5 e 6.

Bissecao	Newton	Secante	Exato
2.3250000000	2.3333333330	2.3333333333	2.3333333333
2.3375000000	2.3333333330	2.3333333333	2.3333333333
2.3312500000	2.3333333330	2.3333333333	2.3333333333
2.3343750000	2.3333333330	2.3333333333	2.3333333333
2.3328125000	2.3333333330	2.3333333333	2.3333333333
2.3335937500	2.3333333330	2.3333333333	2.3333333333
2.3332031250	2.3333333330	2.3333333333	2.3333333333
2.3333984375	2.3333333330	2.3333333333	2.3333333333
2.3333007813	2.3333333330	2.3333333333	2.3333333333

Tabela 4. Resultados das aproximações para a Raiz 1

Bissecao	Newton	Secante	Exato
7.6750000000	7.6666666649	7.6666666667	7.6666666667
7.6625000000	7.6666666649	7.6666666667	7.6666666667
7.6687500000	7.6666666649	7.6666666667	7.6666666667
7.6656250000	7.6666666649	7.6666666667	7.6666666667
7.6671875000	7.6666666649	7.6666666667	7.6666666667
7.6664062500	7.6666666649	7.6666666667	7.6666666667
7.6667968750	7.6666666649	7.6666666667	7.6666666667
7.6666015625	7.6666666649	7.6666666667	7.6666666667
7.6666992188	7.6666666649	7.6666666667	7.6666666667

Tabela 5. Resultados das aproximações para a Raiz 2

Bissecao	Newton	Secante	Exato
9.3250000000	9.3333333341	9.3333333334	9.3333333334
9.3375000000	9.3333333341	9.3333333333	9.3333333334
9.3312500000	9.3333333341	9.3333333333	9.3333333334
9.3343750000	9.3333333341	9.3333333333	9.3333333334
9.3328125000	9.3333333341	9.3333333333	9.3333333334
9.3335937500	9.3333333341	9.3333333333	9.3333333334
9.3332031250	9.3333333341	9.3333333333	9.3333333334
9.3333984375	9.3333333341	9.3333333333	9.3333333334
9.3333007813	9.3333333341	9.3333333333	9.3333333334

Tabela 6. Resultados das aproximações para a Raiz 3

Os três métodos apresentaram resultados eficazes na determinação das raízes da função. O método de Newton-Raphson, como esperado, convergiu rapidamente, mas o

método da secante se destacou ao obter resultados muito próximos ao valor exato em menos iterações, sugerindo que, para esta função específica, a aproximação da derivada feita pela secante foi vantajosa. O método da bisseção, embora mais lento, forneceu uma solução robusta, com precisão controlada pela tolerância especificada.